DSA STUDY BLUEPRINT SERIES

58. Flatten a Doubly Linked List Leetcode 430

Flattening Nested Child-Pointer Doubly Linked Lists

Section 07 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Flatten DLL nodes containing `child` pointer branches.
- Traverse lists; when child node exists, append to tail and update pointers.

Memory & Execution Blueprint

1 ↔ 2 ↔ 3 (with child 7 ↔ 8) ⇒ Flattened: 1 ↔ 2 ↔ 3 ↔ 7 ↔ 8

C++ Implementation Reference

Standard code layout and syntax

```
// Flatten: DFS/BFS traversal of node branches.
// For each node with a child, recursively link the child list
// between the current node and its original next node.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	O(N)
Space Complexity	O(1) (Recursive stack if DFS)

Key Practices & Gotchas

- **Important:** Cut child node pointers after merging to prevent malformed DLL definitions.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace the re-wirings of three node pointers reversing segment `10 -> 20`:

Step	Pointers [prev, curr, next]	Pointer reassignment	Resulting node state
1	[nullptr, 10, 20]	`curr->next = prev`	10 points to nullptr
2	[10, 20, nullptr]	`curr->next = prev`	20 points to 10 (Reversed segment completed!)

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Pointer Re-linking:** Modifying node transitions requires dummy nodes to isolate head pointer alterations.
- **Fast & Slow Pointers:** Useful for loop check detections, middle-node finding, and cyclic lists splits.
- **Related LeetCode Problems:** #206 Reverse Linked List, #141 Linked List Cycle, #23 Merge k Sorted Lists.